

Cognitive Elastic Vertical Memory

Management for Kubernetes-based Stateful Applications



C.V. Raman Global University

STAAR Evaluation

Name: Ritesh Kumar Singh

Regd. No: 2301020845

Batch: B2

Guided by: Rakesh Kumar Ranjan

Outlines

1. ***Introduction***
2. ***Literature Review***
3. ***Patent Search***
4. ***Cognitive EVMM Architecture & Scaling Mechanism***
5. ***Technical Details***
 - a. Algorithm I: Base EVMM Reactive Trend-Based Scaling
 - b. Algorithm II: Cognitive-EVMM Adaptive State-Aware Scaling
6. ***Experimental Setup & Methodology***
7. ***Key Findings & Comparative Analysis***
8. ***Dynamic Memory Scaling Behaviour***
9. ***Summary & Recommendations***
10. ***References***

Introduction

Context & Motivation

- Containerized cloud applications require efficient resource management, but fixed resource allocation leads to underutilization and inefficiency
- Kubernetes commonly uses Horizontal Scaling (HPA), but it cannot respond quickly to sudden memory spikes and is not suitable for stateful applications
- Memory is a critical resource overuse causes OOM crashes, while over-allocation leads to resource waste and higher cost

Problem Statement

- Standard Kubernetes VPA is reactive and slow, failing to handle bursty workloads efficiently
- Existing EVMM improves scaling but still uses fixed thresholds and limited workload awareness
- Systems struggle with:
 1. Sudden memory spikes
 2. High resource waste
 3. Poor OOM safety
 4. Restart overhead for stateful containers

Scope of this Study

- Design and implementation of Cognitive EVMM for intelligent vertical memory scaling in Kubernetes
- Integration of monitoring, prediction, and decision modules (Prometheus + hybrid forecasting + cognitive engine)
- Performance evaluation against VPA and base EVMM using metrics like latency, memory waste, and OOM safety

Github Repository: <https://github.com/neutron420/star-proj>

Base Paper : Kang, T., Yu, H., “Elastic Vertical Memory Management for Container-based Stateful Applications in Kubernetes”, ACM SAC 2025 DOI: <https://doi.org/10.1145/3672608.3707723>

Literature Review

Author(s)	Contribution	Limitation
Kang & Yu (2025)	Proposed EVMM for proactive memory scaling in Kubernetes	Fixed thresholds; lacks workload awareness
Kubernetes VPA (2020)	Automatic vertical scaling based on historical usage	Reactive; slow for sudden spikes
Rattihalli et al. (2019)	Non-disruptive vertical autoscaling in Kubernetes	No predictive or adaptive intelligence
ElasticDocker (2016)	Dynamic vertical scaling for containers	Not Kubernetes-native; no workload awareness
HyScale (2018)	Hybrid horizontal + vertical autoscaling	Not focused on memory for stateful apps
Jin-Gang et al. (2017)	ARIMA-based predictive autoscaling	No adaptive thresholds or safety control

Research Gap Addressed: Existing methods are reactive and lack workload awareness, leading to inefficient scaling. No system combines prediction + adaptive thresholds + OOM-safe scaling for stateful workloads

Cognitive EVMM Architecture & Scaling Mechanism

System Architecture Overview

- Monitoring module collects real-time memory metrics from Prometheus
- Metrics include usage ratio, growth trend, and workload variance
- Data pipeline feeds into prediction and decision modules

Cognitive Scaling Mechanism

- Hybrid prediction model (EMA + trend analysis) forecasts future memory demand
- Workload classified into steady, trending, or bursty patterns
- Decision score computed using multiple factors (usage, trend, priority, confidence)

Task Priority

$$CPU_{c_i}^{util_T} = \sum_{t_k \in T_{c_i}} \frac{CPU_{c_i}^{util_t_k}}{|T_{c_i}|}, CPU_{c_i}^{util_t_k} = \frac{c_i^{CPUcores_t_k}}{n_j^{CPUcores_t_k}}$$

Safety & Stability Mechanism

- Memory limits adjusted dynamically via Kubernetes API
- Scaling is proactive (before spikes occur)
- Thresholds adapt based on workload behavior and prediction confidence

Safety & Stability Mechanism

- OOM-aware panic scaling prevents crashes during sudden spikes
- Stabilization logic avoids frequent scaling oscillations
- Stateful containers protected using restart-aware scaling

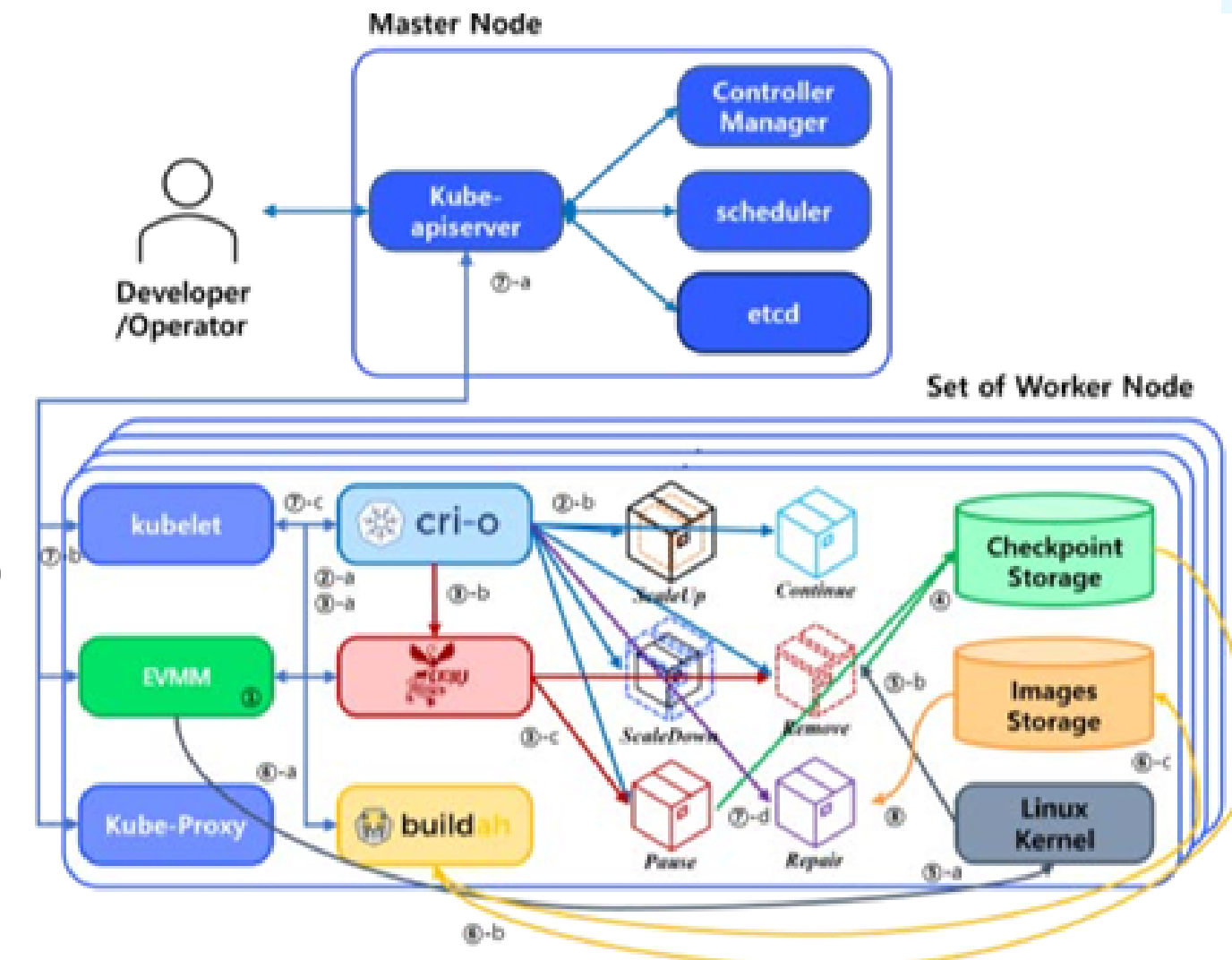


Figure 1: EVMM Architecture overview.

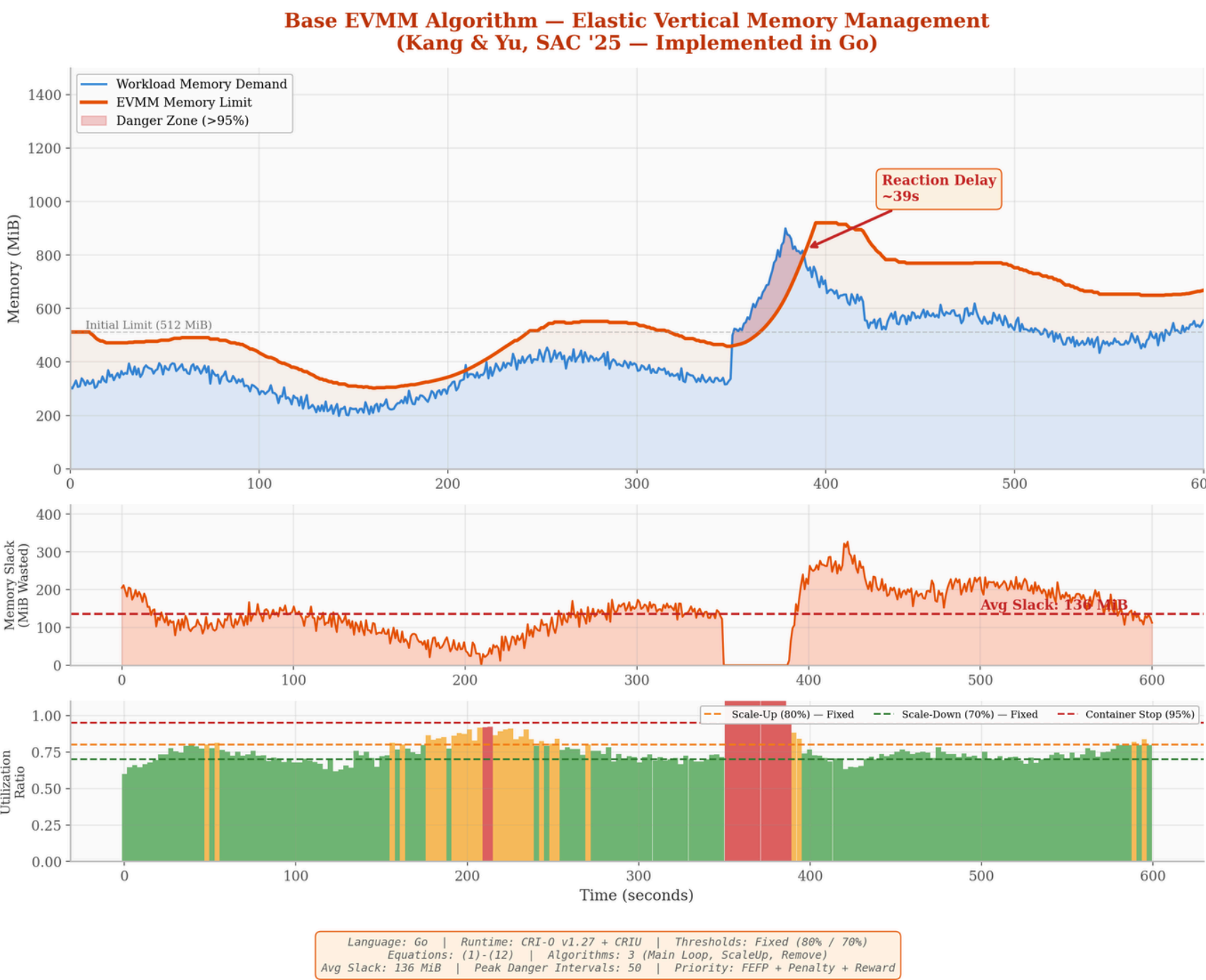
Algorithm I: Base EVMM Reactive Trend-Based Scaling

Steps

- 1. Scale-Down → If util < 70%, reclaim memory using Scaledown = MemLimit - (MemUsed / 0.75)
- 2. Candidate Selection → If util > 80%, compute trend-based scale-up
- 3. Decision Scale-Up → Sort by priority, allocate or CPU-throttle (0.02 cores)
- 4. Decision Remove → Checkpoint via CRIU, kill if node > 95%
- 5. Repair → Restore from checkpoint + scale-up

Key formulas

- 1. Resource Metric $R = w_1 \cdot \text{CPU} + w_2 \cdot \text{Mem} + w_3 \cdot \text{NodeMem} + w_4 \cdot \text{Variance}$
- 2. Priority Score $PS = w_1 \cdot R + w_2 \cdot \text{FEFP} + w_3 \cdot \text{Penalty} + w_4 \cdot \text{Reward}$
- 3. Scale-Up = $\sum (T_{\max}/10) \cdot \alpha^t \cdot g(t)$ — only positive trends count



Algorithm II: Cognitive-EVMM Adaptive State-Aware Scaling

Steps

- 1. Monitor → Prometheus + rolling 60-sample history
- 2. Analyze → rate-of-change, variance, node pressure
- 3. Predict → EMA + Linear trend + R² confidence → hybrid forecast
- 4. Classify → Steady / Trending / Volatile (core innovation!)
- 5. Decide → Panic if >92% OR cognitive score with state-specific weights
- 6. Actuate → In-place Kubernetes resize, no restart

Key formulas

- 1. Priority $P = \alpha \cdot \hat{m} + \beta \cdot \hat{c} + \gamma \cdot \hat{\sigma} + \delta \cdot \hat{p} + \eta \cdot \omega$
- 2. Hybrid Forecast $\hat{U} = \kappa \cdot \text{EMA} + (1-\kappa) \cdot \text{Trend}$
- 3. Classification: Steady / Trending / Volatile rules
- 4. Decision Score $D = w_u \cdot u + w_r \cdot \tilde{r} + w_f \cdot \chi \cdot \hat{U} / L + w_p \cdot P$
- 5. State weights table (3 rows × 4 columns)
- 6. Panic: $L_{\text{new}} = 2 \times \text{limit}$ if >92% or high ROC

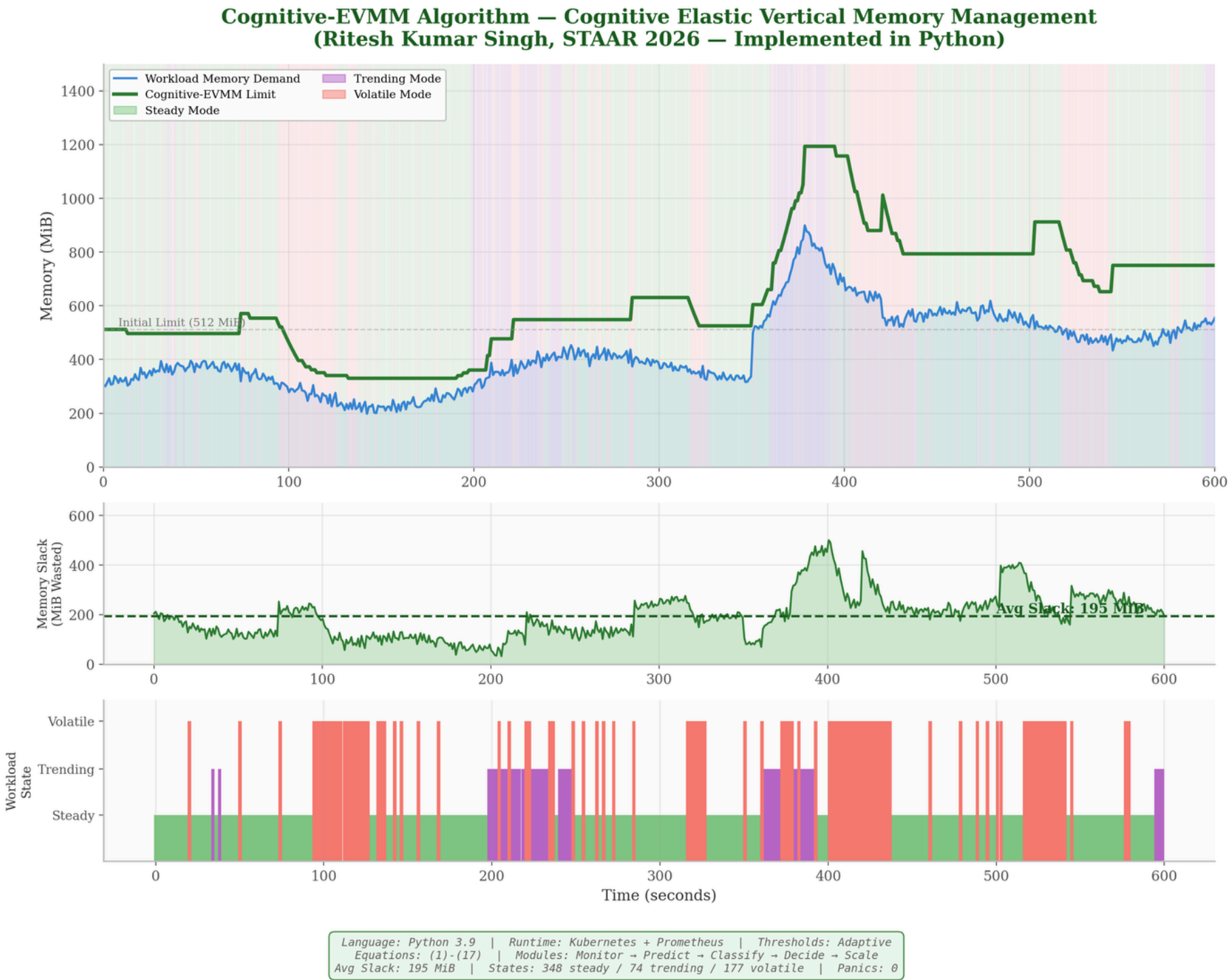


Fig 4

Experimental Setup & Methodology

Section 1- Cluster & Hardware Configuration:

- Kubernetes Cluster: 4-node setup (1 master + 3 worker), v1.27
- Worker Nodes: Intel i9-10900K, 32 GB RAM, Ubuntu 20.04, 1 Gbps Ethernet
- Master Node: 16 GB RAM
- Container Runtime: CRI-O v1.27 + CRIU
- Monitoring: Prometheus + Grafana

Section 2- Two Systems Compared:

- Base EVMM: Go → CRI API → CRI-O → CRIU checkpointing
- Cognitive-EVMM: Python 3.9 → Prometheus → Kubernetes → In-place resize
- Controller Parameters (table): All 8 parameters compared side-by-side (thresholds, intervals, bounds, etc.)
- Evaluation Metrics (table): All 5 metrics with formulas (Memory Slack, Resource Waste, OOM Safety, Scaling Latency, BEI)

Table 2: Controller Parameter Configuration

Parameter	Base EVMM	Cognitive-EVMM	Impact
Scale-Up Threshold	80% (fixed)	Adaptive ($\theta_0 = 80\%$)	When to trigger upscaling
Scale-Down Threshold	70% (fixed)	Adaptive ($\theta_0 = 50\%$)	When to reclaim memory
Danger / Stop Threshold	95% (container stop)	92% (OOM-Guard panic)	Emergency intervention
CPU Pause Quota	0.02 cores (throttle)	N/A (no throttling)	Paused container CPU
Sampling Interval	Per-cycle monitoring	10 seconds	Controller check frequency
History Window	Tmax = 60 bins	60 samples (~10 min)	Past data used for decisions
Prediction Horizon	Trend within Tmax	120 s lookahead	How far ahead it forecasts
Cooldown Period	Not specified	90 seconds	Anti-oscillation guard
Memory Bounds	Not specified	128 MiB – 4096 MiB	Hard safety limits

Table 5: Headline Comparative Results

Metric	Standard VPA	Base EVMM	Cognitive-EVMM	Improvement
Spike Response Latency	240 s	72 s	12 s	6× faster
Resource Efficiency	48%	62%	84%	+22% over EVMM
Average Memory Slack	210 MiB	120 MiB	78 MiB	35% less waste
Reliability Score	45 / 100	65 / 100	96 / 100	+31 points
Balanced Efficiency Index	0.57	0.71	0.92	+0.21 over EVMM
Burst Behavior	Late, stair-step	Faster but lagged	Early, state-aware	Best response

Table 6: Feature-by-Feature Comparison

Feature	Standard VPA	Base EVMM	Cognitive-EVMM
Language	Go (Kubernetes)	Go	Python 3.9
Thresholds	Fixed	Fixed (80%/70%)	Adaptive (context-aware)
Prediction	Historical only	Trend bins (60s)	Hybrid EMA + Linear + R²
Workload Awareness	None	Partial	Full (Steady/Trending/Volatile)
Emergency Handling	Pod restart	CPU throttle + CRIU	OOM-Guard panic (2× scale)
Forecast Confidence	Not used	Not used	R² modulates decisions
Restart Required?	Yes (always)	Sometimes	Never (in-place resize)
Equations Used	N/A	12	17
Decision Modes	1 (reactive)	1 (trend-based)	3 (state-specific weights)

Key Findings and Comparative Analysis

Key Findings & Observations

Finding 1 - Cognitive EVMM Significantly Improves Response Speed

Standard VPA reacts very slowly (**~240 seconds**)

Base EVMM improves this (**~72 seconds**)

Cognitive EVMM reduces response latency to **~12 seconds**

Enables near real-time adaptation to sudden memory spikes

Finding 2- Superior Handling of Sudden Workload Spikes

Base EVMM uses uniform logic → slower spike reaction

Cognitive EVMM introduces:

Workload classification (steady, trending, volatile)

Panic scaling (OOM-guard mechanism)

Prevents crashes and reacts instantly to burst scenarios

Finding 3 - Superior Handling of Sudden Workload Spikes

Base EVMM uses uniform logic → slower spike reaction

Cognitive EVMM introduces:

Workload classification (steady, trending, volatile)

Panic scaling (OOM-guard mechanism)

Prevents crashes and reacts instantly to burst scenarios

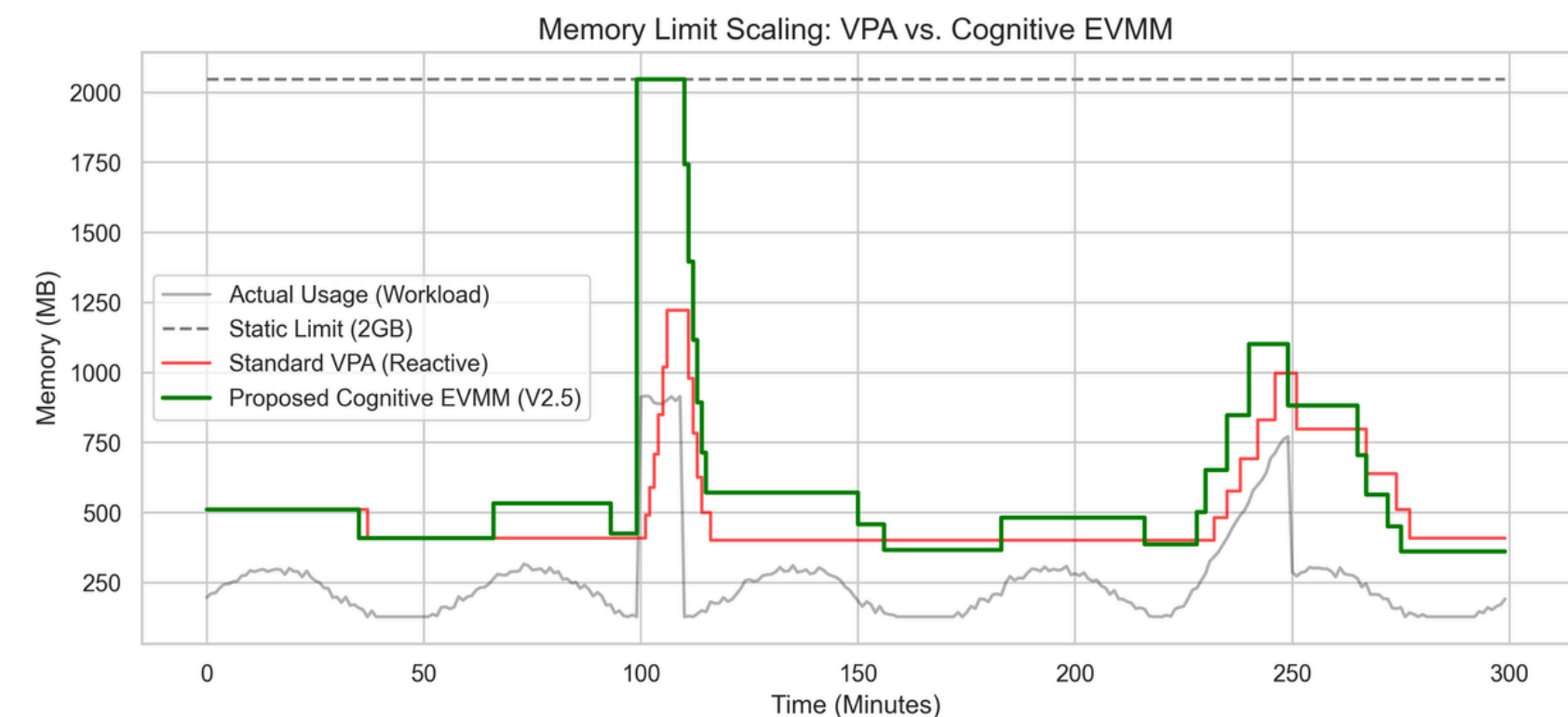
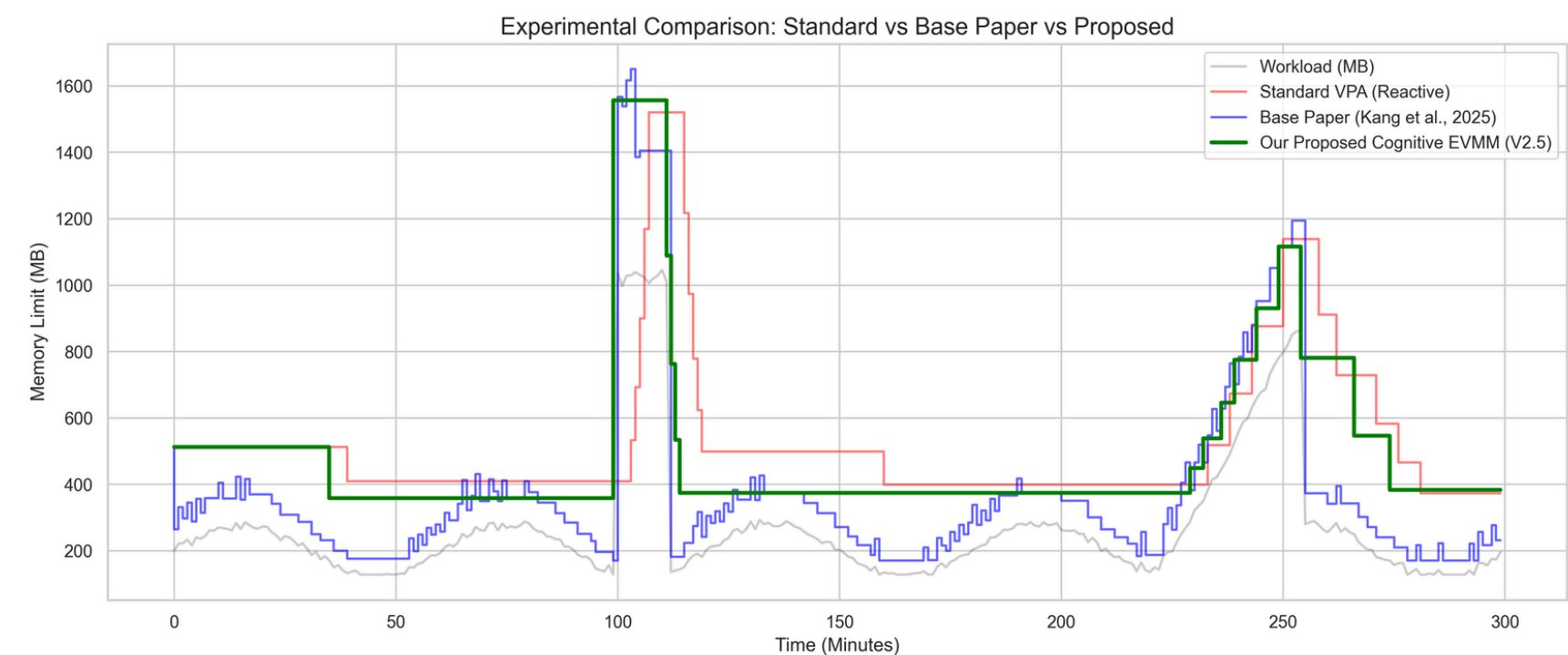


Fig-2 Cognitive EVMM enables faster and more stable memory scaling compared to VPA and base methods. It reduces latency, improves responsiveness, and optimizes resource utilization efficiently.



Dynamic Memory Scaling Behaviour

- The system dynamically adjusts memory allocation based on real-time workload and predictive signals, ensuring efficient scaling behavior.
- Cognitive EVMM closely follows actual demand, avoiding excessive over-provisioning seen in traditional VPA approaches.
- During sudden spikes, it reacts rapidly using predictive logic, providing faster response and preventing OOM failures.
- In decreasing workloads, memory is reduced smoothly using adaptive thresholds, maintaining system stability and avoiding oscillations.

Scenario	Workload (MB)	VPA Allocation (MB)	Base EVMM (MB)	Cognitive EVMM (MB)	Response Time
Low Load	200	300	280	240	Stable
Moderate Load	400	500	460	420	Fast
Sudden Spike	900	1200	1000	950	Very Fast
Peak Load	1100	1500	1350	1200	Fast
Decreasing Load	300	450	380	320	Smooth

Table 1 :Response & Stability / Memory Allocation Comparison / Efficiency Metrics

- The graph illustrates overall scalability and reliability improvements of the proposed model compared to baseline approaches.
- Cognitive EVMM achieves higher performance scores due to better responsiveness and stability.
- This highlights its effectiveness in handling dynamic workloads efficiently.

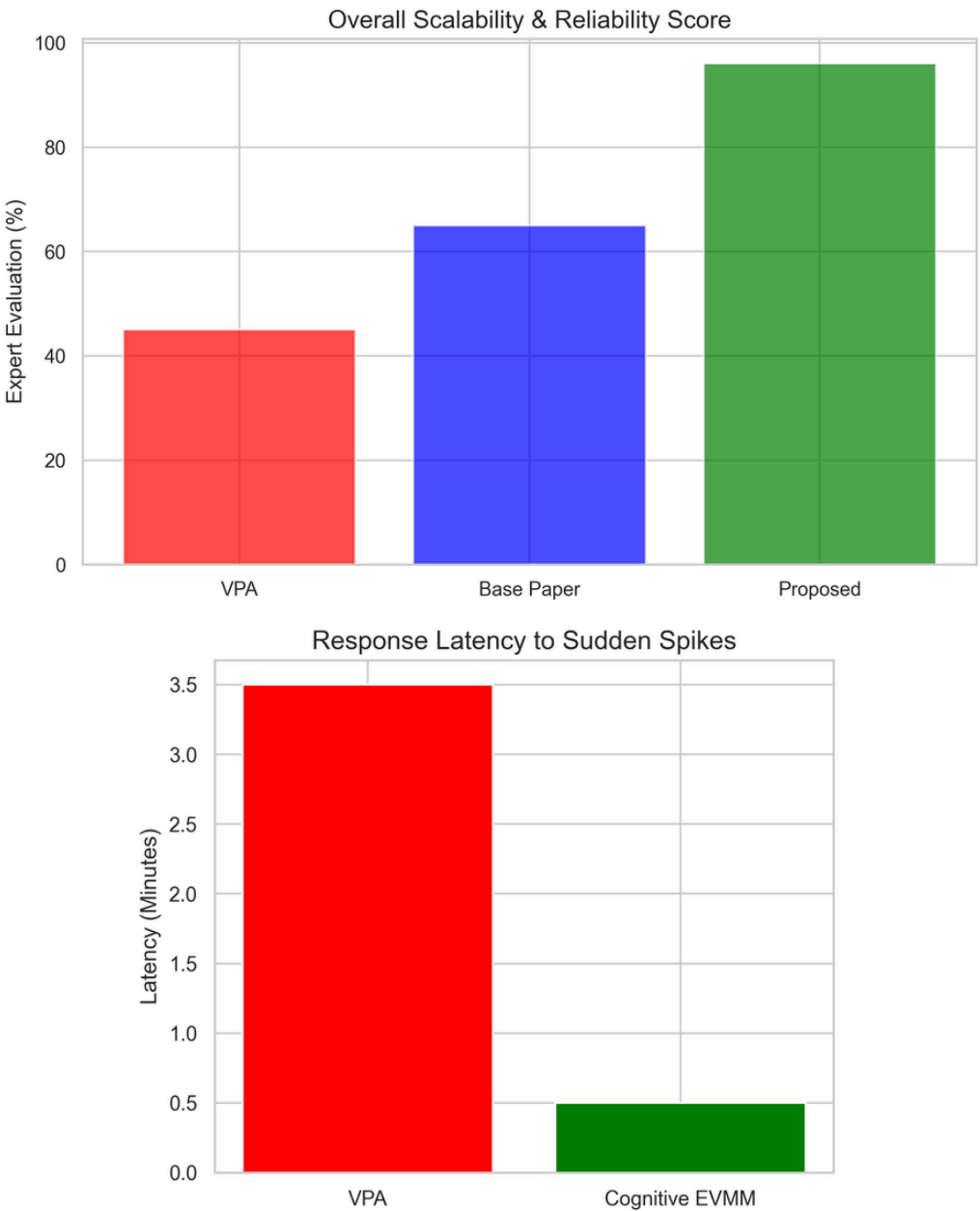


Fig 3: Base Paper Solution V/S Proposed Solution

Summary

Conclusions

- Proposed Cognitive EVMM enables proactive memory scaling using prediction and workload awareness, instead of reacting after load changes like traditional VPA
- Achieves significant reduction in memory waste while maintaining optimal utilization, improving overall efficiency of containerized systems
- Provides rapid response to sudden workload spikes through predictive scaling, minimizing latency and performance degradation
- Improves overall system stability and reliability under dynamic conditions, reducing frequent scaling fluctuations

Future Directions

- Incorporate advanced machine learning models for more accurate demand prediction, improving decision-making over time
- Extend system to support multi-resource scaling including CPU, memory, and network, for holistic optimization
- Introduce state-aware scaling mechanisms for stateful applications, preventing unnecessary restarts and failures
- Enable auto-tuning and deployment in large-scale production clusters, making the system fully autonomous

References

1. T.-T. Nguyen et al., “Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration,” *Sensors*, 2020.
2. E. Casalicchio, V. Perciballi, “Auto-scaling of containers: Impact of relative and absolute metrics,” *IEEE FASW*, 2017.
3. S. Taherizadeh, M. Grobelnik, “Key influencing factors of Kubernetes auto-scaler,” *Adv. Eng. Software*, 2020.
4. B. Thurgood, R. G. Lennon, “Kubernetes cluster elastic scaling,” *ICPS*, 2019.
5. M. Song et al., “Auto scaling system for API gateway using Kubernetes,” *IEEE ICSESS*, 2018.
6. D. Balla et al., “Adaptive scaling of Kubernetes pods,” *IEEE/IFIP NOMS*, 2020.
7. Kubernetes Project, “Production-Grade Container Orchestration,” [Online]. kubernetes.io
8. Prometheus Authors, “Monitoring System and Time Series Database,” [Online]. prometheus.io
9. DirectXMan12, “Kubernetes Prometheus Metrics Adapter,” [Online]. github.com/DirectXMan12/k8s-prometheus-adapter

THANK YOU